

Wording for *boolean-testable*

Document #: P1964R2
Date: 2020-02-11
Project: Programming Language C++
LWG
Reply-to: Tim Song
<t.canens.cpp@gmail.com>

Contents

- [1 Introduction](#)
- [2 Revision history](#)
- [3 LEWG Belfast vote](#)
- [4 Drafting notes](#)
- [5 Wording](#)
- [6 References](#)

§ 1 Introduction

This paper provides wording for replacing `boolean` with the *boolean-testable* exposition-only concept, as proposed in [[P1964R0](#)]. For detailed motivation and discussion, see that paper.

§ 2 Revision history

- R2: Incorporated LWG feedback in Prague.
- R1: Initial draft of wording.

§ 3 LEWG Belfast vote

An early draft of [[P1964R0](#)] was presented to LEWG on Friday afternoon in Belfast. The following polls were taken:

Introduce an exposition-only concept `$NAME` as a replacement for the existing use of the `boolean` concept.

- subsume `convertible_to`
- add syntactic checking for `!`
- add semantic requirements for “there is no `operator ||` nor `operator&&` that can be found in any use of `T`”

In favor of the above design.

SF	F	N	A	SA
3	11	2	0	0

Name is *boolean-testable*

Unanimous consent

Tim and Casey will update P1964 with wording for the above design. We forward that paper to LWG for C++20.

Unanimous consent

§ 4 Drafting notes

Yes, this is a lot of words to specify what is basically “don’t be dumb”. Defining “dumb” with sufficient precision turns out to be complicated.

The basic requirement is that a *boolean-testable* type must not contribute any potentially viable `operator&&` or `operator||` overload to the overload set. If so, then any two *boolean-testable* types can be freely used with these operators, because there are no viable user-defined overloads and therefore they must have their built-in meaning.

There are three cases to consider:

1. Members. These are relatively straightforward: no `operator&&` or `operator||`, period.
2. Non-member functions. These are also simple: we know the concrete types, so they are either potentially viable (an implicit conversion sequence exists from the source expression to one of the parameter types) or they are not.
3. Non-member function templates. These are the hardest because we don’t know the parameter types at all and must therefore resort to template argument deduction.

There are some wrinkles for this third case.

§ Non-deduced contexts

Consider the following example:

```
namespace X {
    enum A {};

    template<class>
    struct trait { using type = A; };

    template<class T>
    void operator&&(T*, typename trait<T>::type);
}
```

This `operator&&` will be picked up in an expression like `(int*) nullptr && x::A()`. Since `int*` should model *boolean-testable* (and certainly isn't responsible for this operator), we must make `x::A` not model *boolean-testable*. In other words, a non-deduced context, which can encode an arbitrary type transformation, must be considered to match everything.

§ `std::valarray`

As mentioned in [P1964R0], the wording needs to avoid catching overloads like `std::valarray`'s `operator&&` on unsuspecting types in namespace `std`. That is, we want declarations like

```
template<class T>
valarray<bool> operator&&(const valarray<T>&, const typename
    valarray<T>::value_type&);
```

and its pre-[LWG3074] form

```
template<class T>
valarray<bool> operator&&(const valarray<T>&, const T&);
```

to disqualify specializations of `std::valarray` (or any derived class that might have added a conversion to `bool`), but not `std::true_type`.

The distinguishing characteristics of these overloads are

1. They are part of the interface of some class template (e.g., `std::valarray`);
2. They have a function parameter whose (uncvref'd) type is a specialization of that class template;
3. They are a member of the same namespace as that class template (so that they can be found by ADL)

For these overloads, we can safely only consider the parameter(s) satisfying #2 above (called *key parameters* in the wording below). This is because for template argument deduction to succeed on such a parameter, the type of the provided argument must be either a specialization of that class template or derived from it. If so, then *that argument must necessarily also bring this function template into the overload set*. As long as the wording excludes such arguments, then, there is no need to worry about other types that may happen to belong in the same namespace.

I have color-coded the two steps in this reasoning because there are corner cases involving each, which I'll now discuss.

§ Non-deduced contexts, again

Consider this example:

```
namespace Y {
    template<class>
    struct C {};

    template<class T>
    void operator&&(C<T> x, T y);
```

```

template<class T>
void operator||(C<std::decay_t<T>> x, T y);

enum A {};
}

struct B { template<class T> operator T(); };

```

We don't want the `operator&&` declaration to disqualify `Y::A`; however the expression `::B() || Y::A()` will use the `Y::operator||` overload, and therefore `Y::A` cannot be *boolean-testable*, even though its declaration might be superficially similar. The key difference here is that `C<std::decay_t<T>>` doesn't contain anything that participates in template argument deduction, and therefore no longer requires the argument to have any relation to `C`, contrary to the [first sentence of the reasoning above](#).

To qualify as a *key parameter*, then, the type must contain at least one template parameter that participates in template argument deduction.

§ Hidden friends

Hidden friends strike at the [second sentence of the reasoning above](#). Consider:

```

namespace Z {
    template<class>
    struct A {
        operator bool();
    };

    struct B {
        operator bool();
        template<class T>
        friend void operator&&(A<T>, B);
    };
}

```

`Z::A<int>() && Z::B()` will use the `operator&&` overload, but ADL for `Z::A<int>()` alone will not even find the hidden friend overload (indeed, the author of `Z::A` might have nothing to do with it). So we must disqualify `Z::B` instead.

The problem here is that the hidden friend can be a friend of the wrong class. That means that the [second sentence of the reasoning above](#) no longer applies, because we are no longer guaranteed that the argument related to `A` will bring the overload in.

The wording below excludes hidden friends from the *key parameter* special case: hidden friends disqualify a type from modeling *boolean-testable* if template argument deduction for either parameter succeeds. Note that the concern motivating this special case doesn't apply to hidden friends: if they are the hidden friend of the “right” class template, then ADL for other types in the namespace will not even find them, so they will not accidentally disqualify anything.

§ 5 Wording

This wording is relative to [N4842].

Replace 18.5.2 [concept.boolean] with the following:

§ 18.5.2 Boolean testability [concept.booleantestable]

- 1 The exposition-only *boolean-testable* concept specifies the requirements on expressions that are convertible to `bool` and for which the logical operators (7.6.14 [expr.log.and], 7.6.15 [expr.log.or], 7.6.2.1 [expr.unary.op]) have the conventional semantics.

```
template<class T>
concept boolean-testable-impl = convertible_to<T, bool>; // exposition only
```

- 2 Let `e` be an expression such that `decltype((e))` is `T`. `T` models *boolean-testable-impl* only if:

- .1) — either `remove_cvref_t<T>` is not a class type, or name lookup for the names `operator&&` and `operator||` within the scope of `remove_cvref_t<T>` as if by class member access lookup (11.8 [class.member.lookup]) results in an empty declaration set; and
- .2) — name lookup for the names `operator&&` and `operator||` in the associated namespaces and entities of `T` (6.5.2 [basic.lookup.argdep]) finds no disqualifying declaration (defined below).

- 3 A *disqualifying parameter* is a function parameter whose declared type `P`

- .1) — is not dependent on a template parameter, and there exists an implicit conversion sequence (12.4.3.1 [over.best.ics]) from `e` to `P`; or
- .2) — is dependent on one or more template parameters, and either
 - .1) — `P` contains no template parameter that participates in template argument deduction (13.10.2.5 [temp.deduct.type]), or
 - .2) — template argument deduction using the rules for deducing template arguments in a function call (13.10.2.1 [temp.deduct.call]) and the type of `e` as the argument type succeeds.

- 4 A *key parameter* of a function template `D` is a function parameter of type `cv X` or reference thereto, where `X` names a specialization of a class template that is a member of the same namespace as `D`, and `X` contains at least one template parameter that participates in template argument deduction.

[Example: In

```
namespace Z {
    template<class>
    struct C {};

    template<class T>
    void operator&&(C<T> x, T y);

    template<class T>
    void operator||(C<type_identity_t<T>> x, T y);
}
```

the declaration of `Z::operator&&` contains one key parameter, `C<T> x`, and the declaration of `Z::operator||` contains no key parameters. — *end example*]

5 A *disqualifying declaration* is

- .1) — a (non-template) function declaration that contains at least one disqualifying parameter; or
- .2) — a function template declaration that contains at least one disqualifying parameter, where
 - .1) — at least one disqualifying parameter is a key parameter; or
 - .2) — the declaration contains no key parameters; or
- .3) — the declaration declares a function template that is not visible in its namespace (9.8.1.2 [namespace.memdef]).

6 [*Note*: The intention is to ensure that given two types `T1` and `T2` that each model *boolean-testable-impl*, the `&&` and `||` operators within the expressions `declval<T1>() && declval<T2>()` and `declval<T1>() || declval<T2>()` resolve to the corresponding built-in operators. — *end note*]

```
template<class T>
concept boolean-testable = // exposition only
    boolean-testable-impl<T> && requires (T&& t) {
        { !std::forward<T>(t) } -> boolean-testable-impl;
    };

```

7 Let `e` be an expression such that `decltype((e))` is `T`. `T` models *boolean-testable* only if `bool(e) == !bool(!e)`.

8 [*Example*: The types `bool`, `true_type` (20.15.2 [meta.type.synop]), `int*`, and `bitset<N>::reference` (20.9.2 [template.bitset]) model *boolean-testable*. — *end example*]

Edit 18.3 [concepts.syn] as indicated:

```
namespace std {
    [...]

    // 18.5, comparison concepts
    - // 18.5.2, concept boolean
    - template<class B>
    - concept boolean = see below ;

    [...]
}

```

Replace all instances of `boolean` in 17.11.4 [cmp.concept], 18.5.3 [concept.equalitycomparable], 18.5.4 [concept.totallyordered] and 18.7.4 [concept.predicate] with *boolean-testable*.

Edit 16.4.2.1 [expos.only.func] as indicated:

```
constexpr auto synth-three-way =
    [<class T, class U>(const T& t, const U& u)
    requires requires {
    - { t < u } -> convertible_to<bool>;
    }

```

```

- { u < t } -> convertible_to<bool>;
+ { t < u } -> boolean-testable;
+ { u < t } -> boolean-testable;
}
{
  if constexpr (three_way_comparable_with<T, U>) {
    return t <== u;
  } else {
    if (t < u) return weak_ordering::less;
    if (u < t) return weak_ordering::greater;
    return weak_ordering::equivalent;
  }
};

```

Edit 25.5.5 [\[alg.find\]](#) p1 as indicated:

- 1 Let E be:
 - (1.1) — `*i == value` for `find`,
 - (1.2) — `pred(*i) != false` for `find_if`,
 - (1.3) — `pred(*i) == false` for `find_if_not`,
 - (1.4) — `bool{invoke(proj, *i) == value}` for `ranges::find`,
 - (1.5) — `bool{invoke(pred, invoke(proj, *i))} != false` for `ranges::find_if`,
 - (1.6) — `bool{!invoke(pred, invoke(proj, *i))} == false` for `ranges::find_if_not`.

Edit 25.5.7 [\[alg.find.first.of\]](#) p1 as indicated:

- 1 Let E be:
 - (1.1) — `*i == *j` for the overloads with no parameter `pred`,
 - (1.2) — `pred(*i, *j) != false` for the overloads with a parameter `pred` and no parameter `proj1`,
 - (1.3) — `bool{invoke(pred, invoke(proj1, *i), invoke(proj2, *j))} != false` for the overloads with parameters `pred` and `proj1`.

Edit 25.5.8 [\[alg.adjacent.find\]](#) p1 as indicated:

- 1 Let E be:
 - (1.1) — `*i == *(i + 1)` for the overloads with no parameter `pred`,
 - (1.2) — `pred(*i, *(i + 1)) != false` for the overloads with a parameter `pred` and no parameter `proj`,
 - (1.3) — `bool{invoke(pred, invoke(proj, *i), invoke(proj, *(i + 1)))} != false` for the overloads with parameters `pred` and `proj`.

Edit 25.5.9 [\[alg.count\]](#) p1 as indicated:

1 Let E be:

- (1.1) — `*i == value` for the overloads with no parameter `pred` or `proj`,
- (1.2) — `pred(*i) != false` for the overloads with a parameter `pred` but no parameter `proj`,
- (1.3) — `invoke(proj, *i) == value` for the overloads with a parameter `proj` but no parameter `pred`,
- (1.4) — `bool(invoke(pred, invoke(proj, *i))) != false` for the overloads with both parameters `proj` and `pred`.

Update the value of `__cpp_lib_concepts` in 17.3.2 [version.syn], header `<version>` synopsis, to reflect the date of approval of this proposal.

§ 6 References

[LWG3074] Jonathan Wakely. Non-member functions for `valarray` should only deduce from the `valarray`.
<https://wg21.link/lwg3074>

[N4842] Richard Smith. 2019. Working Draft, Standard for Programming Language C++.
<https://wg21.link/n4842>

[P1964R0] Tim Song. 2019. Casting `convertible_to<bool>` considered harmful.
<https://wg21.link/p1964r0>